

Hive vs Pig: Querying and Cleaning Unstructured Data at Scale

BDA POP — Session 09/22/2026

Why this session exists

Last week's session established that Hadoop's HDFS+YARN pair removes the single-machine ceiling, and that MapReduce is the programming model that runs on top of it — but it also named Hive and Pig only in passing, as tools that “compile down to MapReduce so nobody writes it by hand.” That's true, but it skips the part that actually matters day to day: what does an analyst asking a question, or an engineer cleaning yesterday's raw logs, actually type? This week answers that directly. Hive gives a SQL-shaped front end over data already sitting in HDFS — the same partition-pruning and table-design choices a database administrator makes, applied to a cluster instead of one server. Pig gives a scripting front end for the messier job that has to happen first: turning raw, half-broken order and delivery logs into the clean, structured rows Hive can query in the first place. Both exist because “write a Map function and a Reduce function” is the wrong level of abstraction for either job.

STEP 0: LIBRARIES

```
# Base R only this week -- everything below (split, aggregate, table) ships  
# with R itself, no install.packages() needed on posit.cloud.
```

STEP 1: THE RUNNING DATASET, EXTENDED TO ORDER-LEVEL LOGS

Same self-generated Zomato/Swiggy restaurant dataset as previous sessions — `set.seed(42)`, no external files. This week adds two things a Hive/Pig session actually needs: a `city` column (so we have something realistic to partition by) and an `orders` table generated *from* the restaurants — many order-log rows per restaurant, the way a real platform's raw event stream would arrive, with a deliberate slice of dirty rows mixed in.

```
set.seed(42)  
n_restaurants <- 200  
  
restaurant_data <- data.frame(  
  restaurant_id = 1:n_restaurants,  
  platform = sample(c("Zomato", "Swiggy"), n_restaurants, replace = TRUE),  
  average_cost_two = round(runif(n_restaurants, min = 150, max = 2500)),  
  distance_center_km = round(runif(n_restaurants, min = 0.5, max = 15), 1),  
  rating = round(runif(n_restaurants, min = 1.5, max = 4.9), 1),  
  votes = round(runif(n_restaurants, min = 10, max = 1500))
```

```

)

# More votes + higher cost => higher estimated revenue (linear relationship)
restaurant_data$estimated_revenue <- round(
  (restaurant_data$votes * 1.5) + (restaurant_data$average_cost_two * 0.4) -
  (restaurant_data$distance_center_km * 50) + rnorm(n_restaurants, mean = 500, sd =
100)
)
restaurant_data$estimated_revenue[restaurant_data$estimated_revenue < 100] <- 100

# Expensive restaurants are more likely to offer online delivery (binary relationship)
prob_delivery <- 1 / (1 + exp(-(-1 + 0.0015 * restaurant_data$average_cost_two)))
restaurant_data$has_online_delivery <- rbinom(n_restaurants, 1, prob_delivery)

# Price tier from cost (non-linear rule relationship)
restaurant_data$price_range <- factor(cut(restaurant_data$average_cost_two,
                                          breaks = c(0, 400, 800, 1500, Inf),
                                          labels = c("Budget", "Moderate", "Premium", "Luxury")))

# --- EXTENSION 1: a city per restaurant -- this is the column we'll PARTITION BY ---
cities <- c("Bangalore", "Mumbai", "Delhi", "Hyderabad", "Chennai", "Pune")
restaurant_data$city <- sample(cities, n_restaurants, replace = TRUE)

# --- EXTENSION 2: order-level logs, many rows per restaurant, spanning 14 days ---
# Real platforms don't hand HIVE one row per restaurant -- they hand it millions of
# raw order-event rows. We simulate a small slice of that: 15-40 orders per
# restaurant, each with its own date and delivery time, plus a deliberately dirty
# slice (~4%) with a missing or negative delivery time -- exactly the kind of
# garbage a raw ingestion pipeline produces before anyone cleans it.
order_id <- 1L
order_rows <- list()
for (r in 1:n_restaurants) {
  n_orders <- sample(15:40, 1)
  order_dates <- as.character(Sys.Date() - sample(0:13, n_orders, replace = TRUE))
  base_time <- 20 + restaurant_data$distance_center_km[r] * 2
  delivery_time <- round(rnorm(n_orders, mean = base_time, sd = 6))
  order_rows[[r]] <- data.frame(
    order_id = order_id:(order_id + n_orders - 1),
    restaurant_id = r,
    city = restaurant_data$city[r],
    order_date = order_dates,
    delivery_time_minutes = delivery_time
  )
  order_id <- order_id + n_orders
}
orders <- do.call(rbind, order_rows)

```

```
# Inject dirty rows: ~4% of raw log rows get a missing or nonsensical delivery time.
# This is what "unstructured/raw log data" actually looks like before a Pig-style
# cleaning pass -- not free text, just untrustworthy fields mixed into the stream.
n_orders_total <- nrow(orders)
dirty_idx <- sample(seq_len(n_orders_total), size = round(0.04 * n_orders_total))
half <- length(dirty_idx) %/% 2
orders$delivery_time_minutes[dirty_idx[seq_len(half)]] <- NA
orders$delivery_time_minutes[dirty_idx[(half + 1):length(dirty_idx)]] <- -1

cat("Restaurants:", n_restaurants, " | Order-log rows:", n_orders_total,
    " | Cities:", length(cities), " | Date span: 14 days\n")
```

```
## Restaurants: 200 | Order-log rows: 5401 | Cities: 6 | Date span: 14 days
```

```
head(orders, 5)
```

```
##   order_id restaurant_id      city order_date delivery_time_minutes
## 1         1             1 Bangalore 2026-09-23                 23
## 2         2             1 Bangalore 2026-09-14                 30
## 3         3             1 Bangalore 2026-09-17                 22
## 4         4             1 Bangalore 2026-09-21                 35
## 5         5             1 Bangalore 2026-09-16                 18
```

Hive: a SQL front end over data that never leaves HDFS

Hive doesn't store data itself — it sits on top of HDFS (last week's storage layer) and gives it a schema and a query language. Four pieces do the work:

- **Driver** — receives the HiveQL statement, manages the session and the query's lifecycle from submission to result.
- **Compiler** — parses the query, checks it against the schema the Metastore holds, and turns it into an execution plan: historically a MapReduce job, increasingly a Tez or Spark DAG.
- **Metastore** — the piece that makes Hive feel like a database: a separate relational store (often its own small MySQL/Derby instance) holding table definitions, column names and types, and which HDFS directory each table's (and each partition's) files actually live in. Hive itself never touches raw HDFS bytes until a query needs them — schema lookups hit the Metastore, not the data.
- **Execution Engine** — runs the compiled plan against the cluster and streams results back.

```
# A stand-in for what the Metastore actually stores: not the data, just the
# schema and the on-disk location pattern for each table.
metastore <- data.frame(
  table_name = c("orders"),
  columns = c("order_id:int, restaurant_id:int, city:string, order_date:string, delivery_time_minutes:int"),
  partitioned_by = c("order_date"),
  hdfs_location = c("/warehouse/orders/order_date=<date>/")
)
print(metastore, row.names = FALSE)
```

```
## table_name
## orders
##
## columns
## order_id:int, restaurant_id:int, city:string, order_date:string, delivery_time_minutes:int
## partitioned_by hdfs_location
## order_date /warehouse/orders/order_date=<date>/
```

The location pattern matters: `order_date=<date>` in the path is Hive’s **partitioning** — each partition value gets its own HDFS subdirectory, so a query naming a partition in its `WHERE` clause can skip every directory that isn’t that one, without reading a single row from them.

Managed vs. external tables

```
table_kinds <- data.frame(
  Kind = c("Managed (internal)", "External"),
  `Hive owns the data?` = c("Yes -- DROP TABLE deletes the HDFS files too",
    "No -- Hive only owns the schema/metadata"),
  `Typical use` = c("Data Hive itself created or fully controls",
    "Raw files another pipeline (e.g. Flume, Sqoop) already writes to HDFS"),
  check.names = FALSE
)
print(table_kinds, row.names = FALSE)
```

```
## Kind Hive owns the data?
## Managed (internal) Yes -- DROP TABLE deletes the HDFS files too
## External No -- Hive only owns the schema/metadata
## Typical use
## Data Hive itself created or fully controls
## Raw files another pipeline (e.g. Flume, Sqoop) already writes to HDFS
```

Getting this choice wrong is a common production mistake: pointing a *managed* table at a shared raw log directory means one careless `DROP TABLE` in Hive deletes files other pipelines still depend on. External tables exist specifically so Hive can query data it doesn't own the lifecycle of.

Partitioning and bucketing: Hive's two data-layout tools

Partitioning splits a table's files by a column's value (here, `order_date`) into separate HDFS directories, so a query that filters on that column reads only the matching directories. **Bucketing** goes further *within* a partition: it hashes a column into a fixed number of files, so joins and sampling on that column don't need to scan the whole partition either.

```
# Simulate Hive's partition directories with split() -- one "directory" per
# order_date, exactly matching the metastore's partitioned_by column above.
orders_by_date <- split(orders, orders$order_date)
target_date <- names(orders_by_date)[1]
```

```
full_scan_rows <- nrow(orders)
partition_scan_rows <- nrow(orders_by_date[[target_date]])
```

```
cat("Query: rows for city='Bangalore' on order_date =", target_date, "\n")
```

```
## Query: rows for city='Bangalore' on order_date = 2026-09-10
```

```
cat("Rows a NAIVE scan of the whole table would read:", full_scan_rows, "\n")
```

```
## Rows a NAIVE scan of the whole table would read: 5401
```

```
cat("Rows a PARTITION-PRUNED scan actually reads:", partition_scan_rows, "\n")
```

```
## Rows a PARTITION-PRUNED scan actually reads: 361
```

```
# Bucketing example: hash restaurant_id into 8 buckets within the table.
# A join or sample on restaurant_id only needs to touch the matching bucket
# file, not every partition's full contents.
n_buckets <- 8
orders$bucket <- orders$restaurant_id %% n_buckets
bucket_counts <- table(orders$bucket)
cat("Rows per bucket (should be roughly even across", n_buckets, "buckets):\n")
```

```
## Rows per bucket (should be roughly even across 8 buckets):
```

```
print(bucket_counts)
```

```
##
## 0 1 2 3 4 5 6 7
## 685 684 673 674 649 626 692 718
```

Metric	Value	Threshold	Status
Naive full-table scan vs. one partitioned query	~14x more rows read	should be 1 partition's worth	Flagged
Bucket row-count spread (max vs. min bucket)	within a few % of even	roughly even across buckets	Within threshold

Reading the panel: a query that names `order_date` in its `WHERE` clause and hits a partitioned table reads roughly a fourteenth of the data a naive scan would — one day's directory instead of all fourteen. That's the same “don't move what you don't have to” principle as last week's map-local computation, applied to query planning instead of job scheduling.

Hive execution modes: Local vs. MapReduce

Hive picks between two execution modes, and which one runs is about the size of the job, not the query itself:

Mode	Used when	What actually runs
Local mode	One data node, a small input (a quick test query against a sample)	Runs in a single JVM process, no cluster job submitted
MapReduce mode	Multiple data nodes, large input — this week's 5,401-row <code>orders</code> table at real platform scale would be billions of rows	Compiles to a MapReduce (or Tez) job spread across the cluster

The same `SELECT ... GROUP BY` below would run in Local mode against a laptop-sized sample while an engineer is still writing it, then switch to MapReduce mode with zero syntax changes once pointed at the full production table — the mode is a runtime decision Hive makes, not something the HiveQL itself declares.

Getting data into a Hive table

Before any `SELECT` runs, the table has to exist and hold data. This is the DDL/DML half of “Data Processing” that a query-only example skips over:

```
# DDL/DML an analyst would type to stand up this week's table and load a raw
# file into it -- shown as text, since this session has no live Hive cluster.
hiveql_setup <- "
CREATE DATABASE IF NOT EXISTS delivery_ops;
USE delivery_ops;

CREATE TABLE orders (
  order_id INT, restaurant_id INT, delivery_time_minutes INT
)
PARTITIONED BY (order_date STRING);

LOAD DATA INPATH '/raw/orders_2026_09_06.csv'
INTO TABLE orders PARTITION (order_date='2026-09-06');
"
cat(hiveql_setup)
```

```
##
## CREATE DATABASE IF NOT EXISTS delivery_ops;
## USE delivery_ops;
##
## CREATE TABLE orders (
##   order_id INT, restaurant_id INT, delivery_time_minutes INT
## )
## PARTITIONED BY (order_date STRING);
##
## LOAD DATA INPATH '/raw/orders_2026_09_06.csv'
## INTO TABLE orders PARTITION (order_date='2026-09-06');
```

LOAD DATA INPATH doesn't copy bytes twice — it *moves* the file from its raw HDFS location straight into the partition directory the Metastore already expects (`order_date=2026-09-06/` , matching the metastore table shown earlier). Nothing is parsed or validated at load time; the schema is only checked when a query reads the data back, which is exactly why the dirty rows in this dataset load successfully and only show up as a problem once someone runs `AVG()` over them.

Querying and joining in HiveQL

```
# The HiveQL an analyst would actually type -- shown as text since this session
# has no live Hive cluster, with the equivalent result computed in R below so
# you can see the shape of the answer it returns.
hiveql_query <- sprintf("
SELECT city, COUNT(*) AS n_orders, AVG(delivery_time_minutes) AS avg_delivery
FROM orders
WHERE order_date = '%s'
GROUP BY city;
", target_date)
cat(hiveql_query)
```

```
##
## SELECT city, COUNT(*) AS n_orders, AVG(delivery_time_minutes) AS avg_delivery
## FROM orders
## WHERE order_date = '2026-09-10'
## GROUP BY city;
```

```
# R equivalent, run against the (still-dirty) orders table for one partition
one_day <- orders_by_date[[target_date]]
hiveql_result <- aggregate(delivery_time_minutes ~ city, data = one_day,
                           FUN = function(x) c(n = length(x), avg = round(mean(x, n
a.rm = TRUE), 1)))
print(hiveql_result)
```

```
##           city delivery_time_minutes.n delivery_time_minutes.avg
## 1 Bangalore                67.0                31.9
## 2  Chennai                 50.0                36.8
## 3   Delhi                 78.0                36.2
## 4 Hyderabad               74.0                33.7
## 5   Mumbai                 41.0                32.4
## 6    Pune                  41.0                32.2
```

Notice the NA-driven fragility here: `AVG()` over a column with missing or negative values gives a misleading answer (or requires a defensive `WHERE delivery_time_minutes IS NOT NULL` on every query that touches it). Hive can query dirty data, but it shouldn't have to — which is exactly the job the next tool exists for.

A second, equally common HiveQL shape is a **JOIN** against a second table, plus one of Hive's built-in functions (`ROUND()`, alongside `FLOOR()` and `CEILING()`):

```
# HiveQL an analyst would type to break delivery time down by price tier --
# joining the order log to the restaurant dimension table by restaurant_id.
hiveql_join <- sprintf("
SELECT r.price_range, ROUND(AVG(o.delivery_time_minutes), 1) AS avg_delivery
FROM orders o
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
WHERE o.order_date = '%s'
GROUP BY r.price_range;
", target_date)
cat(hiveql_join)
```

```
##
## SELECT r.price_range, ROUND(AVG(o.delivery_time_minutes), 1) AS avg_delivery
## FROM orders o
## JOIN restaurants r ON o.restaurant_id = r.restaurant_id
## WHERE o.order_date = '2026-09-10'
## GROUP BY r.price_range;
```

```
# R equivalent: merge() is a JOIN, aggregate() is the GROUP BY, round() is ROUND().
joined <- merge(one_day, restaurant_data[, c("restaurant_id", "price_range")],
               by = "restaurant_id")
join_result <- aggregate(delivery_time_minutes ~ price_range, data = joined,
                        FUN = function(x) round(mean(x, na.rm = TRUE), 1))
print(join_result)
```

```
##   price_range delivery_time_minutes
## 1      Budget                35.6
## 2    Moderate                34.9
## 3     Premium                34.5
## 4      Luxury                32.7
```

The JOIN itself looks exactly like a SQL join because it is one — HiveQL doesn't invent new syntax for combining two tables, it borrows SQL's.

The data-quality problem Hive alone doesn't solve

```
n_invalid <- sum(is.na(orders$delivery_time_minutes) | orders$delivery_time_minutes <
0)
pct_invalid <- round(100 * n_invalid / nrow(orders), 1)
cat("Raw order-log rows with a missing or negative delivery_time_minutes:",
    n_invalid, "/", nrow(orders), "(", pct_invalid, "%)\n")
```

```
## Raw order-log rows with a missing or negative delivery_time_minutes: 216 / 5401 (
4 %)
```

Metric	Value	Threshold	Status
Raw log rows with invalid delivery_time_minutes	~4%	0% (every row should be usable)	Flagged

Reading the panel: every `AVG(delivery_time_minutes)` query above is already quietly wrong by a few percent of its input, and a query without the defensive `IS NOT NULL` filter would be far worse — a `-1` value pulls a city's average down without any error being raised. This is precisely the kind of raw-log mess an ingestion pipeline (real platforms use Flume or Sqoop, see below) hands to Hadoop every day, and it needs a cleaning step *before* Hive, not a workaround inside every query written against it.

Pig: a scripting front end for the cleanup pass

Hive is built for someone who already knows what question they want answered. Pig is built for someone who needs to describe a multi-step transformation — load, filter, group, join, store — as a short, readable script, without hand-writing MapReduce for each step. Pig Latin (Pig’s language) reads like a pipeline, not a single query, which is exactly the shape “turn today’s raw order log into a clean table” takes.

Pig Latin’s data model: Atom, Tuple, Bag, Map

Where Hive’s data model is tables split into partitions and buckets, Pig Latin builds everything from four smaller pieces:

```
# Atom -- a single primitive value (int, float, string). One field.
atom_example <- orders$delivery_time_minutes[1]
cat("Atom:", atom_example, "\n")
```

```
## Atom: 23
```

```
# Tuple -- an ordered sequence of fields, same as one row in an RDBMS.
tuple_example <- orders[1, c("order_id", "restaurant_id", "city", "delivery_time_minu
tes")]
cat("Tuple (one order row):\n"); print(tuple_example, row.names = FALSE)
```

```
## Tuple (one order row):
```

```
## order_id restaurant_id      city delivery_time_minutes
##           1              1 Bangalore                23
```

```
# Bag -- a collection of tuples, same as a table. Every data frame we've
# built this session (orders, clean_orders, restaurant_data) IS a bag.
bag_example <- orders[1:3, c("order_id", "city", "delivery_time_minutes")]
cat("\nBag (a small collection of tuples):\n"); print(bag_example, row.names = FALSE)
```

```
##
## Bag (a small collection of tuples):
```

```
## order_id      city delivery_time_minutes
##           1 Bangalore                23
##           2 Bangalore                30
##           3 Bangalore                22
```

```
# Map -- key-value pairs, key always a string, value any type. A named list
# is R's equivalent: one order represented as field-name -> value.
map_example <- list(order_id = orders$order_id[1], city = orders$city[1],
                    delivery_time_minutes = orders$delivery_time_minutes[1])
cat("\nMap (one order, keyed by field name):\n"); str(map_example)
```

```
##
## Map (one order, keyed by field name):
```

```
## List of 3
## $ order_id          : int 1
## $ city              : chr "Bangalore"
## $ delivery_time_minutes: num 23
```

A Pig relation like `raw_orders` in the script below is, under the hood, a **bag of tuples** — which is why `FILTER` (keep some tuples), `GROUP` (bucket tuples into new bags by a key), and `FOREACH ... GENERATE` (build a new tuple per group) are the only verbs Pig Latin really needs: they're all operations on bags and tuples.

Pig execution modes: Local vs. MapReduce

Pig chooses between the same two kinds of mode as Hive, but the Local mode means something slightly different — it talks to the plain Linux filesystem, not HDFS:

Mode	Reads/writes from	Used when
Local mode	The local Linux filesystem	Testing a script against a small sample before pointing it at the cluster
MapReduce mode (default)	HDFS, via jobs on the Hadoop cluster	Production runs — Pig Latin compiles to MapReduce jobs, same engine Hive uses

Pig runs in MapReduce mode by default, which matters for the pipeline below: the `LOAD / FILTER / STORE` script only reads real HDFS paths once it's actually running in that mode.

```
-- The Pig Latin an engineer would actually write for this cleanup job.
-- Not executed here (no live Pig cluster) -- the equivalent runs in R below.
raw_orders = LOAD 'orders_raw.csv' USING PigStorage(',')
              AS (order_id:int, restaurant_id:int, city:chararray,
                  order_date:chararray, delivery_time_minutes:int);

clean_orders = FILTER raw_orders BY delivery_time_minutes IS NOT NULL
                  AND delivery_time_minutes > 0;

by_city      = GROUP clean_orders BY city;

city_stats   = FOREACH by_city GENERATE
              group AS city,
              COUNT(clean_orders) AS n_orders,
              AVG(clean_orders.delivery_time_minutes) AS avg_delivery;

STORE clean_orders INTO '/warehouse/orders_clean' USING PigStorage(',');
```

```
# R equivalent of the Pig Latin above: LOAD -> FILTER -> GROUP -> FOREACH GENERATE.
# `orders` already stands in for the LOAD step (the dataset built in Step 1).
clean_orders <- orders[!is.na(orders$delivery_time_minutes) & orders$delivery_time_mi
minutes > 0, ]

city_stats <- aggregate(delivery_time_minutes ~ city, data = clean_orders,
                        FUN = function(x) c(n = length(x), avg = round(mean(x), 1)))

print(city_stats)
```

```
##           city delivery_time_minutes.n delivery_time_minutes.avg
## 1 Bangalore           955.0           34.0
## 2  Chennai           776.0           37.7
## 3   Delhi          1032.0           35.3
## 4 Hyderabad           964.0           34.3
## 5   Mumbai           752.0           32.4
## 6    Pune           706.0           33.1
```

```
pct_invalid_after <- round(100 * sum(is.na(clean_orders$delivery_time_minutes) |
                                   clean_orders$delivery_time_minutes < 0) / nrow(
clean_orders), 1)
cat("\nRows dropped by the FILTER step:", nrow(orders) - nrow(clean_orders),
    "(", round(100 * (nrow(orders) - nrow(clean_orders)) / nrow(orders), 1), "% )\n")
```

```
##
## Rows dropped by the FILTER step: 216 ( 4 % )
```

```
cat("Invalid delivery_time_minutes remaining in clean_orders:", pct_invalid_after, "%\n")
```

```
## Invalid delivery_time_minutes remaining in clean_orders: 0 %
```

Metric	Value	Threshold	Status
Invalid delivery_time_minutes after the Pig-style FILTER	0%	0%	Within threshold

Reading the panel: the FILTER step is Pig's equivalent of last week's diagnostic pattern — it doesn't make the raw data less messy where it landed, it removes the mess before anything downstream (Hive, in this pipeline) ever queries it. `clean_orders` is what would actually get STORE d into the `/warehouse/orders_clean` directory that a *managed* Hive table would then be pointed at.

```
# Closing the loop: re-run the same HiveQL-shaped query from earlier, now
# against Pig-cleaned data instead of the raw log.
one_day_clean <- clean_orders[clean_orders$order_date == target_date, ]
final_result <- aggregate(delivery_time_minutes ~ city, data = one_day_clean,
                          FUN = function(x) c(n = length(x), avg = round(mean(x),
1)))
print(final_result)
```

```
##      city delivery_time_minutes.n delivery_time_minutes.avg
## 1 Bangalore                67.0                31.9
## 2  Chennai                 50.0                36.8
## 3   Delhi                 78.0                36.2
## 4 Hyderabad                72.0                34.7
## 5   Mumbai                 41.0                32.4
## 6    Pune                  41.0                32.2
```

Hive vs. Pig: same engine underneath, different job on top

```

comparison <- data.frame(
  Aspect = c("Language shape", "Written by", "Best fit", "Data types supported",
    "Runs on the cluster", "Web interface", "Avro support", "Native partitioning"),
  Hive = c("Declarative, SQL-like (HiveQL)", "Analysts who know SQL",
    "\"What's the answer to this question?\"", "Structured data only",
    "Server-side (HiveServer2)",
    "Yes", "No -- not needed, results are queried directly",
    "Yes -- PARTITIONED BY, enforced by the Metastore"),
  Pig = c("Procedural, step-by-step (Pig Latin)", "Data engineers building pipelines",
    "\"Turn this raw mess into that clean table.\", \"Structured, semi-structured, and unstructured\",
    \"Client-side (compiled where the script runs)\",
    \"No\", \"Yes -- output often feeds straight into another language\",
    \"No -- only ad hoc FILTER-based workarounds\")
)
print(comparison, row.names = FALSE)

```

```

##           Aspect
## Language shape      Declarative, SQL-like (HiveQL)
## Written by          Analysts who know SQL
## Best fit            "What's the answer to this question?"
## Data types supported      Structured data only
## Runs on the cluster      Server-side (HiveServer2)
## Web interface          Yes
## Avro support    No -- not needed, results are queried directly
## Native partitioning Yes -- PARTITIONED BY, enforced by the Metastore
##
##           Pig
## Procedural, step-by-step (Pig Latin)
## Data engineers building pipelines
## "Turn this raw mess into that clean table."
## Structured, semi-structured, and unstructured
## Client-side (compiled where the script runs)
## No
## Yes -- output often feeds straight into another language
## No -- only ad hoc FILTER-based workarounds

```

Both still compile down to the same batch execution model (MapReduce, or increasingly Tez) that last week's session covered — the difference is entirely in who's writing the logic and what shape their problem takes, not in what runs on the cluster underneath.

The rest of the ecosystem: getting data in and jobs coordinated

Hive and Pig assume the data is already in HDFS and that someone (or something) runs their jobs on a schedule. Three more tools cover exactly those gaps:

```
ecosystem_tools <- data.frame(
  Tool = c("Sqoop", "Flume", "Oozie", "ZooKeeper"),
  Provides = c("Bulk transfer between HDFS and relational databases",
    "Streaming ingestion of continuous log/event data into HDFS",
    "Workflow scheduling that chains jobs (e.g. Sqoop -> Pig -> Hive) together",
    "Distributed coordination: leader election, shared config, locking"),
  Reached_For_When = c("Nightly-import last month's orders table from a production MySQL database",
    "Continuously stream app/server logs into HDFS as they're generated",
    "\"Every night at 2am: pull new orders, clean with Pig, then refresh the Hive table\"",
    "Multiple cluster services need to agree on who's currently in charge")
)
print(ecosystem_tools, row.names = FALSE)
```

```
##      Tool
##      Sqoop
##      Flume
##      Oozie
## ZooKeeper
##
##                               Provides
##                               Bulk transfer between HDFS and relational databases
##                               Streaming ingestion of continuous log/event data into HDFS
## Workflow scheduling that chains jobs (e.g. Sqoop -> Pig -> Hive) together
##                               Distributed coordination: leader election, shared config, locking
##                               Reached_For_When
n
##                               Nightly-import last month's orders table from a production MySQL database
e
##                               Continuously stream app/server logs into HDFS as they're generated
d
## "Every night at 2am: pull new orders, clean with Pig, then refresh the Hive table"
e"
##                               Multiple cluster services need to agree on who's currently in charge
e
```

Put together, a realistic nightly pipeline reads: **Sqoop/Flume** land raw data in HDFS → **Pig** cleans and reshapes it → a Hive **managed table** exposes the clean result for querying → **Oozie** is what actually ran all three steps in order, unattended, every night → **ZooKeeper** keeps the cluster's own services (YARN's ResourceManager pair, HBase, Kafka where present) agreeing on which instance is currently active. None of these compete with Hive or Pig — each fills a gap neither one was built to cover.

Summary

Check	Result
Naive full-table scan vs. partition-pruned query (rows read)	Flagged — ~14x more rows without partitioning
Bucket row-count spread across 8 buckets	Within threshold — roughly even
Raw order-log rows with invalid delivery_time_minutes	Flagged — ~4% before cleaning
Same check after the Pig-style FILTER step	Within threshold — 0%

Hive’s value is a schema and a query language over data that was always going to live in HDFS; Pig’s value is a readable pipeline for the transformation that has to happen before that data is trustworthy enough to query. Neither replaces MapReduce — both are what most people actually touch instead of it.